

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №5
по дисциплине: БД
тема: «Организация взаимодействия с базой данных через консольное
приложение»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Панченко М. В.

Белгород 2025

Цель работы: получить навыки подключения к различным системам управления базами данных и взаимодействия с ними. Разработать консольное приложение для взаимодействия с базой данных.

Задание к работе:

Вариант 14

1. Подключиться к системе управления базами данных с помощью SQL библиотек выбранного языка программирования.
2. Организовать взаимодействие с базами данных выбранной СУБД (PostgreSQL/MySQL/SQLite).
3. Разработать консольное приложение, в котором производится подключение к базе данных, разработанной на основе предыдущих лабораторных работ, а также обеспечивается выполнение запросов предшествующей лабораторной работы.

Структура моей базы данных из предыдущих работ

```
railway=# \dt
```

Список отношений			
Схема	Имя	Тип	Владелец
public	car	таблица	postgres
public	contactdetails	таблица	postgres
public	passenger	таблица	postgres
public	route	таблица	postgres
public	schedule	таблица	postgres
public	seat	таблица	postgres
public	ticket	таблица	postgres
public	ticketstatus	таблица	postgres
public	train	таблица	postgres

(9 строк)

Код консольного приложения для взаимодействия с БД:

```
import psycopg2
from typing import List, Tuple

DB_CONFIG = {
    "host": "localhost",
    "database": "railway",
    "user": "postgres",
    "password": "postgres",
    "port": 5432,
}

class DatabaseService:
    def __init__(self, config: dict):
        try:
            self.connection = psycopg2.connect(**config)
            self.cursor = self.connection.cursor()
            print("успешное подключение к базе данных\n")
        except psycopg2.Error as e:
            print(f"ошибка подключения к БД: {e}")
            raise

    def execute_query(self, query: str) -> List[Tuple]:
        try:
            self.cursor.execute(query)
            return self.cursor.fetchall()
        except psycopg2.Error as e:
            print(f"ошибка выполнения запроса: {e}")
            return []

    def close(self):
        if self.cursor:
```

```

        self.cursor.close()
    if self.connection:
        self.connection.close()
    print("\nсоединение с БД закрыто")

```

```

class ConsoleInterface:

```

```

    def __init__(self, db: DatabaseService):

```

```

        self.db = db

```

```

        self.queries = {

```

```

            1: ("билеты по типам поездов", self.query_tickets_by_train_type),

```

```

            2: ("средняя цена по маршрутам", self.query_avg_price_by_route),

```

```

            3: ("количество мест по типам вагонов",

```

```

self.query_seats_by_car_type),

```

```

            4: (

```

```

                "популярные направления по сезонам",

```

```

                self.query_popular_routes_by_season,

```

```

            ),

```

```

            5: (

```

```

                "рейтинг поездов по типам вагонов",

```

```

                self.query_train_rating_by_car_type,

```

```

            ),

```

```

        }

```

```

    def query_tickets_by_train_type(self):

```

```

        query = """

```

```

        SELECT t.TrainType, COUNT(tk.ID) as TotalTickets

```

```

        FROM Train t

```

```

        LEFT JOIN Schedule s ON t.ID = s.TrainID

```

```

        LEFT JOIN Car c ON s.ID = c.ScheduleID

```

```

        LEFT JOIN Seat st ON c.ID = st.CarID

```

```

        LEFT JOIN Ticket tk ON st.ID = tk.SeatID

```

```

        GROUP BY t.TrainType;

```

```

        """

```

```

        results = self.db.execute_query(query)

```

```

        self.print_table(["тип поезда", "всего билетов"], results)

```

```

    def query_avg_price_by_route(self):

```

```

        query = """

```

```

        SELECT r.DepartureStation, r.ArrivalStation,

```

```

               ROUND(AVG(tk.Price)::numeric, 2) as AveragePrice

```

```

        FROM Route r

```

```

        JOIN Schedule s ON r.ID = s.RouteID

```

```

        JOIN Car c ON s.ID = c.ScheduleID

```

```

        JOIN Seat st ON c.ID = st.CarID

```

```

        JOIN Ticket tk ON st.ID = tk.SeatID

```

```

        GROUP BY r.DepartureStation, r.ArrivalStation;

```

```

        """

```

```

        results = self.db.execute_query(query)

```

```

        self.print_table(

```

```

            ["станция отправления", "станция прибытия", "средняя цена"], results

```

```

        )

```

```

    def query_seats_by_car_type(self):

```

```

        query = """

```

```

SELECT c.CarType, SUM(c.TotalSeats) as TotalSeatsSum
FROM Car c
JOIN Schedule s ON c.ScheduleID = s.ID
JOIN Route r ON s.RouteID = r.ID
GROUP BY c.CarType;
"""

results = self.db.execute_query(query)
self.print_table(["тип вагона", "всего мест"], results)

def query_popular_routes_by_season(self):
    query = """
SELECT
    CASE
        WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (12, 1, 2) THEN
'Зима'
        WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (3, 4, 5) THEN
'Весна'
        WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (6, 7, 8) THEN
'Лето'
        WHEN EXTRACT(MONTH FROM s.DepartureDateTime) IN (9, 10, 11) THEN
'Осень'
    END AS season,
    r.DepartureStation,
    r.ArrivalStation,
    COUNT(t.ID) AS passengers_count
FROM Ticket t
JOIN Seat st ON t.SeatID = st.ID
JOIN Car c ON st.CarID = c.ID
JOIN Schedule s ON c.ScheduleID = s.ID
JOIN Route r ON s.RouteID = r.ID
GROUP BY season, r.DepartureStation, r.ArrivalStation
ORDER BY season, passengers_count DESC;
"""

    results = self.db.execute_query(query)
    self.print_table(["сезон", "откуда", "куда", "кол-во пассажиров"],
results)

def query_train_rating_by_car_type(self):
    query = """
SELECT
    tr.TrainNumber,
    tr.TrainType,
    c.CarType,
    COUNT(t.ID) AS tickets_sold
FROM Ticket t
JOIN Seat st ON t.SeatID = st.ID
JOIN Car c ON st.CarID = c.ID
JOIN Schedule s ON c.ScheduleID = s.ID
JOIN Train tr ON s.TrainID = tr.ID
GROUP BY tr.TrainNumber, tr.TrainType, c.CarType
ORDER BY c.CarType, tickets_sold DESC;
"""

    results = self.db.execute_query(query)
    self.print_table(

```

```

        ["номер поезда", "тип поезда", "тип вагона", "продано билетов"],
results
    )

    @staticmethod
    def print_table(headers: List[str], rows: List[Tuple]):
        if not rows:
            print("данные отсутствуют\n")
            return

        col_widths = [len(str(h)) for h in headers]
        for row in rows:
            for i, val in enumerate(row):
                col_widths[i] = max(col_widths[i], len(str(val)))

        col_widths = [w + 2 for w in col_widths]

        separator = "+" + "+".join(["-" * w for w in col_widths]) + "+"

        print("\n" + separator)
        header_row = (
            "|"
            + "|".join([str(h).center(w) for h, w in zip(headers, col_widths)])
            + "|"
        )
        print(header_row)
        print(separator)

        for row in rows:
            data_row = (
                "|"
                + "|".join([str(val).ljust(w) for val, w in zip(row,
col_widths)])
                + "|"
            )
            print(data_row)

        print(separator)
        print(f"всего записей: {len(rows)}\n")

    def show_menu(self):
        print("\nсистема железнодорожных касс")
        for num, (name, _) in self.queries.items():
            print(f"[{num}] {name}")
        print("[0] выход")

    def run(self):
        while True:
            self.show_menu()
            try:
                choice = input("\nвыберите запрос: ").strip()

                if choice == "0":
                    print("\nдо свидания")
                    break

```

```
        choice_num = int(choice)
        if choice_num in self.queries:
            print(f"\n{self.queries[choice_num][0]}")
            self.queries[choice_num][1]()
            input("\nнажмите enter для продолжения")
        else:
            print("\nневерный выбор")

    except ValueError:
        print("\nвведите число")
    except KeyboardInterrupt:
        print("\n\nпрограмма прервана пользователем")
        break
```

```
if __name__ == "__main__":
    try:
        db = DatabaseService(DB_CONFIG)

        interface = ConsoleInterface(db)
        interface.run()

        db.close()

    except Exception as e:
        print(f"\nкритическая ошибка: {e}")
    finally:
        print("\n\nпрограмма завершена\n")
```

Результат работы:

```
~/projects/university/5sem/db/lab5 on master !2 ?2 > python 1.py
успешное подключение к базе данных

система железнодорожных касс
[1] билеты по типам поездов
[2] средняя цена по маршрутам
[3] количество мест по типам вагонов
[4] популярные направления по сезонам
[5] рейтинг поездов по типам вагонов
[0] выход

выберите запрос: 1

билеты по типам поездов

+-----+-----+
| тип поезда | всего билетов |
+-----+-----+
| пассажирский | 1 |
| фирменный | 0 |
| скорый | 3 |
+-----+-----+
всего записей: 3

нажмите enter для продолжения2

система железнодорожных касс
[1] билеты по типам поездов
[2] средняя цена по маршрутам
[3] количество мест по типам вагонов
[4] популярные направления по сезонам
[5] рейтинг поездов по типам вагонов
[0] выход

выберите запрос: 2

средняя цена по маршрутам

+-----+-----+-----+
| станция отправления | станция прибытия | средняя цена |
+-----+-----+-----+
| Москва | Казань | 52.00 |
| Москва | Санкт-Петербург | 3500.00 |
+-----+-----+-----+
всего записей: 2
```

Вывод: В процессе выполнения лабораторной работы были получены навыки подключения к различным системам управления базами данных и взаимодействия с ними. Получен опыт в разработке консольного приложения, способного обеспечить подключение к базе данных и выполнение запросов.